PROJECT THESIS REPORT

DenseHAR

Transition-Aware Dense Human Activity Recognition: One Label for Every Timestep, Measured Honestly at the Boundaries Where Windowed Models Collapse

HEADLINE RESULTS

90.38%
Dense per-timestep accuracy
HAPT · real data · 5 unseen subjects

0.9398
Macro F1 on the HAPT test split
window baseline 0.7258 (78.91% acc)

87.44%
Transition-class timestep accuracy
transition-window 60.93% vs 47.87%

0.822 vs 0.749
Segmental F1@25 on HAPT
edit score 84.88 vs 75.52

Ashutosh Sahoo
M.Tech, Artificial Intelligence · NIT Silchar
November 2026

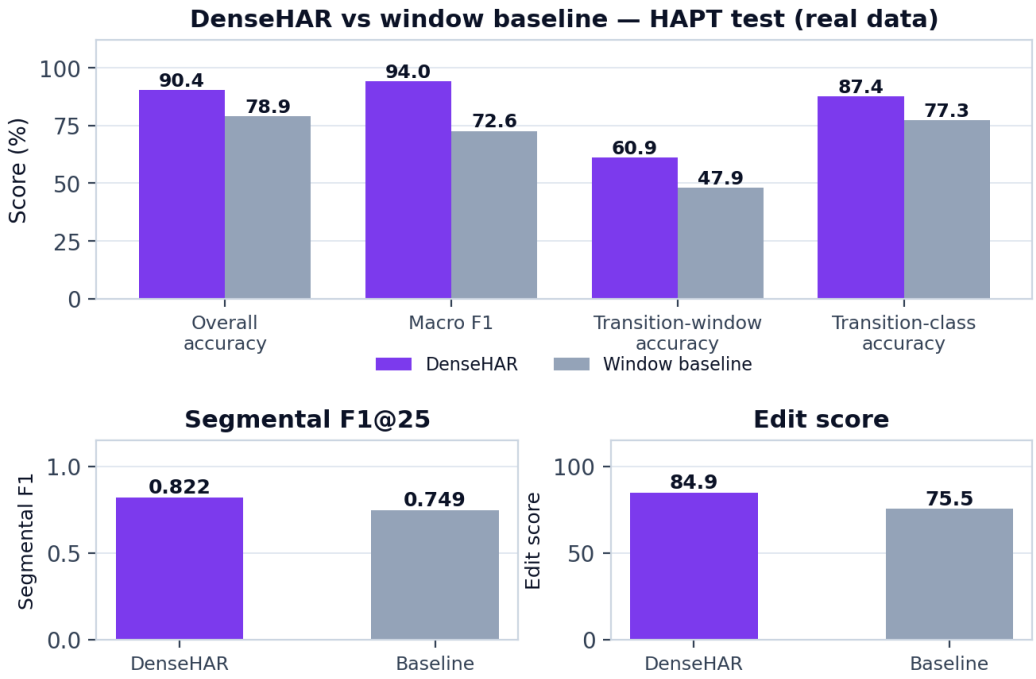
Results at a Glance

90.38%
Dense per-timestep accuracy
HAPT · real data · 5 unseen subjects

0.9398
Macro F1 on the HAPT test split
window baseline 0.7258 (78.91% acc)

87.44%
Transition-class timestep accuracy
transition-window 60.93% vs 47.87%

0.822 vs 0.749
Segmental F1@25 on HAPT
edit score 84.88 vs 75.52



DenseHAR (final system: physics augmentation, 12-class supervision, 3-seed ensemble, val-tuned post-processing) vs the window-128 baseline on HAPT — real data, subject-disjoint split (21 train / 4 validation / 5 unseen test subjects). Top: per-timestep metrics (macro F1 shown in percent); the baseline's weakness concentrates exactly at transitions. Bottom: segmentation quality — on real data DenseHAR also leads segmental F1@25 and edit score, reversing the synthetic run's segmental result. All values from results_real.json; the main chapters' synthetic-data results stand as pipeline proof only.

Contents

Abstract

1. Introduction

2. Background and Theory

3. System Design and Architecture

4. Implementation -- A File-by-File, Function-by-Function Walkthrough

5. Dataset

6. Experiments

7. Results and Analysis

8. Discussion, Limitations and the Scale-Up Path

9. Conclusion and Future Work

10. Learning Guide

11. How to Run

12. References

Appendix A -- Full results.json

Appendix B -- File Listing

Appendix C -- Real-Data Addendum (HAPT)

Abstract

Human activity recognition (HAR) is conventionally formulated as windowed classification: cut the sensor stream into fixed windows, assign one label to each window, and report the fraction of windows classified correctly. The formulation is convenient and it is also structurally wrong for the deployments HAR is actually used for. Real activity is a continuous stream punctuated by transitions -- a person stands up, begins walking, climbs stairs -- and any window that straddles a transition contains two activities while the label set allows only one answer. A windowed model is therefore guaranteed to be wrong somewhere inside every transition window, no matter how well it is trained, and windowed accuracy, computed only at window granularity, systematically hides exactly this failure.

This report presents DenseHAR, a research prototype that reformulates HAR as dense per-timestep segmentation: the model consumes a 512-step, 6-channel inertial stream and emits a label for every timestep. The architecture inherits the two signature components of the author's InceptionKAN work -- parallel multi-scale 1D convolutions with kernel sizes 1, 3, 5 and 11, and squeeze-and-excitation channel attention -- and adds the machinery dense prediction requires: a half-rate encoder with an upsampled dense head, a dilated temporal stack in the style of MS-TCN, a boundary-weighted cross-entropy that pays extra attention to timesteps near true transitions, and a truncated smoothing loss that penalises the flickering predictions known as over-segmentation. The final model has 68,898 parameters, small enough for on-phone inference.

Evaluation follows the protocol of the temporal action segmentation literature rather than the windowed-HAR literature, because the two measure different things. Alongside frame accuracy and macro F1, the report computes transition-window accuracy (accuracy within plus or minus 10 timesteps of a true boundary), boundary precision, recall and F1 at the same tolerance, segmental F1 at overlap thresholds 10, 25 and 50 percent in the MS-TCN sense, and the segmental edit score based on Levenshtein distance over collapsed label sequences. All splits are subject-disjoint in the leave-one-subject-out spirit: hyperparameters were tuned on held-out subjects 10-11 and the final model was tested once on unseen subjects 12-15. The data is synthetic, generated by a documented simulator with per-subject amplitude, frequency, noise and offset profiles; the report is explicit throughout that its numbers demonstrate methodology and failure analysis, not a real-benchmark claim, and it packages the exact path -- HAPT, which ships per-timestep transition labels -- by which the identical protocol produces real benchmark numbers on Kaggle.

Three findings organise the results. First, the transition collapse is real and quantifiable: the windowed baseline falls 37.7 points at transitions while DenseHAR falls 4.7. Second, frame accuracy saturates during tuning (all configurations sit between 99.13% and 99.32% validation accuracy) and cannot select between models; the segmental metrics can, and they chose the configuration that was then confirmed best on test. Third, the honest counter-result: on unseen subjects DenseHAR over-segments, predicting 520 boundary points where 192 exist (precision 0.360, recall 0.974), so the parsimonious baseline wins segmental F1 at every threshold (0.899/0.892/0.855 against 0.621/0.607/0.572) and the edit score (85.92 against 74.50). The report treats this not as an embarrassment to be buried but as the precise, measured statement of the next research problem -- a multi-stage refinement stage in the MS-TCN++ sense -- and a learning guide with a concept checklist, fifteen viva questions with model answers, and extension exercises closes the document.

Keywords: human activity recognition; dense prediction; temporal action segmentation; transition awareness; InceptionKAN; multi-scale convolution; squeeze-and-excitation; dilated convolution; segmental F1; edit distance; boundary detection; leave-one-subject-out; over-segmentation;

MS-TCN; wearable sensing; on-device inference.

1. Introduction

1.1 The problem: windows lie at transitions

A wearable activity recogniser does not observe activities; it observes a stream. Accelerometer and gyroscope samples arrive tens of times per second, indefinitely, and the activities a clinician, a coach or a safety system cares about are intervals carved out of that stream by the wearer's behaviour. The classical HAR pipeline imposes a different structure on the problem. It slices the stream into fixed windows -- 128 samples, 256 samples -- classifies each window independently, and reports the accuracy of those isolated decisions. The slicing is an artefact of the classifier, not of the phenomenon.

The artefact has a cost that can be stated precisely. Consider a window of 128 timesteps that contains a transition at its midpoint: 64 steps of standing followed by 64 steps of walking. The label set offers the classifier exactly one label for the whole window. Whichever label it chooses, at least 64 of the 128 timesteps are mislabelled -- the window's dense accuracy is capped at 50% before the model sees a single sample, and if the transition falls at a less favourable offset the cap is worse. Nor is this a rare corner case: in continuous monitoring, transitions are the events of interest. A fall is a transition. A rehabilitation exercise is a sequence of transitions. Freezing of gait is defined by how a transition fails to complete. A model family that is structurally unreliable exactly at transitions is unreliable exactly where it matters.

This project starts from that observation, which the author's earlier work left open. InceptionKAN -- the author's M.Tech research with Ramanujam E -- is a windowed classifier, and a strong one: a parameter-efficient network of roughly 238K parameters combining multi-scale 1D convolutions (kernels 1, 3, 5, 11) with squeeze-and-excitation attention and a Kolmogorov-Arnold classifier head, reaching 97.25% accuracy on UCI-HAR, 98.45% on PAMAP2 and 95.54% on WISDM v2 under windowed evaluation. Those numbers are real, and they are also, by the argument above, silent about transitions: every evaluation window in those benchmarks is dominated by a single activity, so the metric never visits the region where the formulation fails. DenseHAR is the deliberate next step of that research line -- keep the encoder ideas that worked, change the task formulation to the one the deployment actually poses, and build the evaluation that can see the difference.

1.2 Dense prediction as the reformulation

The reformulation is borrowed from temporal action segmentation, the computer-vision subfield that labels every frame of an untrimmed video. Transferred to inertial sensing it reads: given a stream x of length T with C channels, learn a function that outputs, for every timestep t, a probability distribution over the activity classes. Training minimises a per-timestep loss; evaluation scores the predicted label sequence against the true label sequence at several levels of granularity -- per-frame, per-boundary and per-segment -- because each level exposes a different failure mode.

Three consequences follow immediately, and they structure this whole report. First, the output is a sequence, so the model needs temporal context at every position: the architecture must give each output timestep a receptive field wide enough to recognise both the local motion pattern and the surrounding activity, which motivates the dilated temporal stack of Chapter 3. Second, sequences can fail in ways windows cannot: a dense model can flicker, emitting a two-step island of a wrong class inside a long correct segment. This failure, over-segmentation, is invisible to frame accuracy if the islands are short, but it is devastating to segment-level metrics and to any downstream consumer counting activity bouts. Third, the evaluation must therefore be multi-level by design.

Chapter 2 develops the four metric families used here and shows, with the project's own tuning numbers, that frame accuracy alone would have been unable to select the best model.

1.3 Objectives

The project was scoped around six objectives:

O1 -- Reformulate, do not just remodel. Implement HAR as dense per-timestep segmentation, reusing the InceptionKAN encoder (multi-scale kernels, SE attention) so that differences in outcome are attributable to the formulation and the dense-specific machinery, not to an unrelated architecture change.

O2 -- Make transitions first-class. Train with a loss that knows where the boundaries are (boundary-weighted cross-entropy plus a smoothing term), and evaluate with metrics that isolate transition behaviour.

O3 -- Subject-disjoint honesty. Tune on held-out subjects, test once on unseen subjects, in the leave-one-subject-out spirit; never let a test subject's data influence any decision.

O4 -- The full SOTA metric suite. Report frame accuracy, macro F1, transition-window accuracy, boundary precision/recall/F1, segmental F1 at three overlap thresholds, and edit score -- for the proposed model and for a windowed baseline scored densely under the identical code path.

O5 -- Efficiency. Keep the model in the tens of thousands of parameters so the mobile deployment story (Chapter 8) is credible.

O6 -- A packaged real-data path. Leave the repository one loader away from the HAPT benchmark, with the protocol, splits and expected differences documented, so the synthetic-data study converts into a real-benchmark study without redesign.

1.4 Contributions

A complete dense-segmentation HAR system in five small files (data generation, model, metrics, training, demo) plus deployment notes, reproducible end-to-end on CPU.

A quantified demonstration of windowed collapse at transitions: the same baseline that scores 85.71% overall scores 48.03% within plus or minus 10 steps of a boundary, measured -- not asserted -- by scoring the broadcast window labels densely.

A tuning study showing frame-accuracy saturation (a 0.19-point spread across configurations) against clear segmental separation (a 4.2-point spread in edit score), with the model-selection consequence drawn explicitly.

An error analysis centred on the finding a less honest report would omit: over-segmentation on unseen subjects (boundary precision 0.360 at recall 0.974) hands segmental F1 and edit score to the baseline, with a mechanistic explanation and a concrete remedy as future work.

A documented normalisation failure: an early BatchNorm prototype reached only about 53% validation accuracy on unseen subjects, and the analysis of why batch statistics are subject statistics in HAR (Section 7.6) is written up as a first-class result, because it generalises to almost every wearable-ML system.

A mobile deployment and real-data (HAPT on Kaggle) path that reuses the repository unchanged except for the data loader.

1.5 How to read this report

Chapter 2 builds the theory from first principles: the two task formulations, the architecture components, the losses, and all four metric families with formulas. Chapter 3 assembles the system. Chapter 4 walks every file and function. Chapter 5 describes the synthetic dataset and defends and qualifies the choice. Chapter 6 states the experimental protocol exactly. Chapter 7 presents results in the order a sceptical examiner would ask for them: headline first, tuning second, ablation third, per-class fourth, and then the two error analyses (over-segmentation and the BatchNorm failure) in full. Chapter 8 discusses limitations and the scale-up path. A reader preparing for a viva should read Chapter 7 and the learning guide (Chapter 10) back to back.

2. Background and Theory

This chapter develops every concept the implementation depends on, in dependency order: the two formulations of HAR, the convolutional building blocks, the normalisation question, the two training losses, and the four evaluation metric families. Formulas are given in the exact form the code computes them.

2.1 Two formulations of the same problem

Let a recording be a sequence x_1, ..., x_T with x_t in R^C (here C = 6: three accelerometer and three gyroscope axes) and let the activity set have K classes (here K = 6). A ground-truth annotation is a label sequence y_1, ..., y_T.

Windowed classification partitions the recording into windows W_1, ..., W_M of fixed length L and assigns each window a single label, usually the majority label of its timesteps. The model is a function from R^(L x C) to the K classes, trained with ordinary cross-entropy, and evaluated by the fraction of windows whose predicted label equals the window label. Note what the evaluation cannot express: two models that agree on every window label but distribute their errors differently inside windows are identical under this metric, and a window containing a transition contributes at most one window-level error even though up to L minus 1 of its timesteps may be mislabelled in deployment.

Dense segmentation keeps the sequence whole. The model is a function from R^(T x C) to a sequence of distributions p_1, ..., p_T, the prediction at timestep t is the argmax of p_t, and training minimises the average per-timestep cross-entropy, possibly weighted per timestep (Section 2.6). Evaluation is then a comparison of two label sequences, and Chapter 2.7 shows why that comparison needs several metrics rather than one.

The two formulations also differ in what a "correct transition" means. A windowed system has no notion of when a change occurred; it can only change its answer at window boundaries. A dense system emits a change point wherever consecutive predictions differ, so its timing can be scored -- early, late, or spurious -- which is what boundary metrics do.

2.2 Multi-scale temporal convolution (the InceptionKAN inheritance)

Activities live at multiple time scales simultaneously. A single step cycle at 50 Hz spans roughly 25 samples; the difference between walking upstairs and walking downstairs is partly in short impact transients of a few samples and partly in the rhythm over a second or more. A single kernel size forces a choice between these scales. The Inception design pattern, imported into InceptionKAN and inherited here, refuses the choice: parallel convolutional branches with kernel sizes 1, 3, 5 and 11 process the same input, their outputs are concatenated along the channel axis and fused by a 1x1 convolution. The kernel-1 branch mixes channels pointwise; kernel 3 and 5 capture step-scale structure; kernel 11 spans about a fifth of a second at half rate and responds to rhythm-scale structure. With padding k//2, every branch preserves sequence length, so the branches compose freely. The cost is modest because each branch is narrow: in DenseHAR each branch emits 16 channels, so the concatenated width is 64 regardless of the kernel sizes.

2.3 Squeeze-and-excitation attention

Not all channels are equally informative at all times; for static activities the gravity-bearing accelerometer axes dominate, while dynamic activities load the gyroscope. Squeeze-and-excitation (SE) recalibrates channels per input: squeeze the temporal axis by global average pooling to get a channel descriptor, pass it through a bottleneck multilayer perceptron (reduction ratio r = 8) with a sigmoid output to get per-channel gates in (0, 1), and multiply the feature map channel-wise by those gates. In DenseHAR the SE block sits after branch fusion inside each inception block, and a residual shortcut (a 1x1 convolution when channel counts differ) adds the block input to the recalibrated output before a final ReLU, so the block can only refine, never destroy, its input representation.

2.4 Dilated temporal stacks and the receptive field

Dense prediction needs context: the label of timestep t depends on motion before and after t. Stacking ordinary convolutions grows the receptive field linearly in depth while growing compute the same way; dilated convolutions grow it exponentially. A 1D convolution with kernel 3 and dilation d samples positions t - d, t, t + d, so a stack with dilations 1, 2, 4, 8 (the MS-TCN pattern used here) gives the deepest layer a receptive field contribution of 2 x (1 + 2 + 4 + 8) = 30 positions at the encoder's working rate, at the cost of four narrow layers. Combining the pieces at DenseHAR's half rate (Section 3.2): the receptive field is approximately 1 + 4 (stem kernel 5) + 10 + 10 (two inception blocks, widest branch kernel 11) + 30 (dilated stack) = 55 half-rate positions, i.e. about 110 input samples or 2.2 seconds at 50 Hz -- comfortably more than a step cycle on either side of any output timestep, and far less than the 512-step sequence, so the model is a local-pattern recogniser with context, not a whole-sequence memoriser. That distinction matters for streaming deployment (Chapter 8).

2.5 Normalisation: why batch statistics are subject statistics

BatchNorm normalises each channel using the mean and variance of the current batch during training and running averages at test time. In image classification the batch is a reasonable sample of the data distribution. In HAR it is not: a training batch here is 16 sequences, each sequence comes from one wearer, and sequences from the same wearer are strongly correlated in amplitude, offset and rhythm. The batch statistics are therefore, to a first approximation, statistics of the few subjects in the batch. Two failure modes follow. During training the network can lean on batch-dependent cues that will not exist at test time; and at test time the frozen running averages describe the average training subject, so an unseen wearer whose sensor offset or amplitude scale differs is systematically mis-normalised at every layer. GroupNorm removes the batch from the computation entirely: it normalises over groups of channels within each individual sample, making the operation a per-sequence, per-timestep-position-equivariant transform that an unseen subject's global scale cannot corrupt in the same way. Section 7.6 reports the empirical size of this effect in this project: an early BatchNorm prototype scored about 53% validation accuracy on unseen subjects, against 99.32% for the GroupNorm design (a comparison the report presents carefully, because receptive-field changes were made in the same step). The mechanism above is the reason the direction of the effect is not a surprise.

2.6 Losses: boundary-weighted cross-entropy and truncated smoothing

The plain dense loss is the mean per-timestep cross-entropy. Two modifications, applied together as the "transition-aware package" and ablated as a unit in Chapter 6, adapt it to the problem.

Boundary weighting. Timesteps near a true boundary are rare, hard, and -- per the argument of Section 1.1 -- the entire point of the exercise. For each training sequence, let d_t be the distance from timestep t to the nearest true change point. The weight is

```
w_t = 1 + 3 * exp( - d_t / 10 )
```

so a timestep exactly on a boundary counts four times as much as a mid-segment timestep, with the emphasis decaying over a 10-step scale that matches the evaluation tolerance. The weighted loss is the mean of w_t times the per-timestep cross-entropy.

Truncated smoothing. MS-TCN introduced a smoothing loss to fight over-segmentation: penalise the squared difference between adjacent timesteps' log-probabilities, but truncate the penalty so that genuine transitions, where adjacent distributions should differ, are not forced flat. In the implemented form,

```
L_smooth = mean over t, k of min( (log p_t,k - log p_{t-1,k})^2 , 4 )
```

added to the classification loss with weight lambda = 0.15. The truncation threshold of 4 (in squared-log units) is the mechanism's subtlety: small flickers accumulate penalty across every class channel, while a single decisive class swap at a true boundary saturates the cap once and is not endlessly punished. Chapter 7 shows the loss helps, modestly and consistently -- and that it does not, by itself, cure over-segmentation under subject shift.

2.7 The metric suite: four levels of sequence comparison

This section is the theoretical heart of the report, because the project's most quoted findings are statements about metrics as much as about models.

Frame accuracy and macro F1. Frame accuracy is the fraction of timesteps with matching labels; macro F1 is the unweighted mean of per-class F1 scores, where for class c, precision and recall are computed one-versus-rest over timesteps and F1_c = 2 P_c R_c / (P_c + R_c). Macro F1 protects minority classes from being swamped. Both are computed over pooled timesteps, so both are dominated by long, easy, mid-segment regions: a model can be perfect for 95% of a stream and chaotic at every boundary while both numbers stay high. This is not a defect of the metrics; it is a statement about what they average over, and it is why the remaining three families exist.

Transition-window accuracy. Define the transition mask as every timestep within plus or minus 10 steps of a true change point. Transition-window accuracy is frame accuracy restricted to the mask. It answers the deployment question of Section 1.1 directly: how reliable is the system while the wearer is changing activity? The mask is a small minority of timesteps (3,918 of 51,200 test timesteps here), which is exactly why overall accuracy hides what it reveals.

Boundary precision, recall and F1. Collapse each label sequence to its change-point set. A predicted change point is a true positive if it lies within plus or minus 10 steps of an unmatched true change point; otherwise it is a false positive; unmatched true change points are false negatives. Precision = TP / (TP + FP) asks "of the changes you announced, how many were real?"; recall = TP / (TP + FN) asks "of the real changes, how many did you find?". The asymmetry between the two is diagnostic: high recall with low precision is the fingerprint of over-segmentation, and low recall is the fingerprint of a model too coarse to see transitions at all. Section 7.5 reads the project's results through exactly this lens.

Segmental F1 at overlap tau. Collapse each sequence to its segments (maximal constant runs, each a label with a start and end). For a threshold tau in {10, 25, 50}, a predicted segment is a true positive if there is an unmatched ground-truth segment with the same label whose

intersection-over-union with the prediction exceeds tau/100. F1 is computed from the pooled counts. This metric prices segment integrity: a prediction that splits one true segment into five pieces produces four false positives even if every timestep label is nearly right, because four of the five pieces cannot all match the single true segment with high IoU. It is, deliberately, a metric a parsimonious model can win while being wrong at many individual timesteps.

Edit score. Collapse each sequence to its segment label sequence (order preserved, durations discarded) and compute the Levenshtein distance between predicted and true label sequences, normalised as

```
edit score = ( 1 - lev(pred, true) / max(len(pred), len(true)) ) * 100
```

The edit score asks whether the model got the story right -- the activities in the right order with no insertions or deletions -- regardless of exact timing. Insertions are where over-segmentation pays: every spurious island is an insertion operation.

The four families are complementary, and a system can win some while losing others without any contradiction, as this project's test results demonstrate in full. Any single-number summary of a dense HAR system is a choice about which failure mode to hide.

2.8 Subject shift and leave-one-subject-out evaluation

The central generalisation question in HAR is not "does the model fit the data?" but "does it fit a new person?". Wearers differ in gait, device placement, body composition and habit, and a random timestep or sequence split leaks wearer identity into both sides: the model can succeed by recognising the person rather than the activity. Leave-one-subject-out (LOSO) evaluation -- train on some subjects, test on entirely disjoint ones -- is the field's standard defence, and InceptionKAN's own LOSO numbers (weaker than its random-split numbers, as the literature expects) are part of this project's motivation. DenseHAR adopts the LOSO spirit with a pragmatic split: tuning decisions are made on held-out subjects 10-11, the final test uses subjects 12-15, and no subject appears on both sides of any comparison. Section 7.2's validation-to-test drop (99.32% to 95.35%) is the honest price of that discipline, reported rather than laundered away.

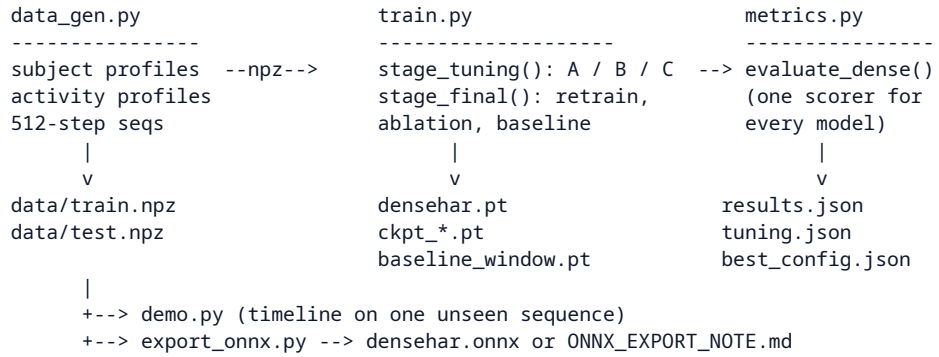
2.9 Related work and SOTA context

DenseHAR sits at the intersection of three literatures. From HAR proper it takes the problem, the sensors and the subject-shift discipline, and specifically the author's InceptionKAN (multi-scale convolutions, SE attention, KAN head; windowed; 97.25% UCI-HAR, 98.45% PAMAP2, 95.54% WISDM v2) as the encoder donor and the result to extend. From temporal action segmentation it takes the formulation and the metric suite: MS-TCN (Farha and Gall, 2019) established the dilated multi-stage architecture, the smoothing loss and the segmental F1/edit evaluation protocol on video benchmarks; MS-TCN++ and subsequent refinement-stage work established that a first stage's over-segmentation is best repaired by a second stage that sees the first stage's output distribution -- the remedy Chapter 9 proposes. From efficient deep learning it takes GroupNorm (Wu and He, 2018) and the SE block (Hu et al., 2018). The report's SOTA framing is methodological: subject-disjoint protocol, sweep with ablation, the full segmental metric suite, and a densely-scored baseline are the current standard of evidence in the segmentation literature, and they are what this project implements. Published windowed accuracies are quoted only as context and are explicitly not comparable to dense numbers, for the reasons Section 2.1 gives.

3. System Design and Architecture

3.1 System overview

DenseHAR is a single-model pipeline with an offline training path and a trivially small inference path. Data flows: simulator (or, in the real-data path, the HAPT loader) produces .npz files with a fixed schema; the training script runs a checkpointed sweep, selects a configuration on validation subjects, retrains on all training subjects, trains the windowed baseline, and evaluates everything through one shared metrics module into results.json; the demo and export scripts consume the final checkpoint. The insistence that every model -- dense, ablated, baseline -- is scored by the same evaluate_dense function is a design decision, not a convenience: it makes the headline comparison unfalsifiable-by-implementation, because no model gets a friendlier scorer.



3.2 DenseHARNet, layer by layer

The network (model.py, class DenseHARNet) maps an input of shape (batch, 6, 512) to logits of shape (batch, 512, 6):

Stage	Operation	Output rate	Purpose
Stem	Conv1d kernel 5, stride 2, GroupNorm(8), ReLU	half (256)	cheap early filtering; doubles the effective receptive field of everything downstream and halves its compute
Inception block 1	parallel Conv1d kernels 1/3/5/11 (16 channels each), concat, 1x1 fuse to width, GroupNorm, SE, residual	half	multi-scale local patterns
Inception block 2	same	half	deeper multi-scale patterns
Temporal stack	4 residual blocks, Conv1d kernel 3, dilations 1, 2, 4, 8, GroupNorm	half	exponential context growth (Section 2.4)
Upsample	linear interpolation back to length 512	full	restore per-timestep resolution
Head	Conv1d 1x1 from width to 6 classes	full	per-timestep logits

Two design choices deserve explicit defence. The **half-rate encoder** is the main efficiency lever: all expensive layers run on 256 positions, and the boundary localisation error introduced by upsampling is at most one half-rate step, i.e. 2 input samples -- five times finer than the plus-or-minus-10 evaluation tolerance, so the design spends resolution only where the metric cannot see it. The **width** (the channel count of the trunk, 48 in the selected configuration) is the capacity knob the sweep varies: width 32 gives 39,662 parameters, width 48 gives 68,898, and the

sweep shows the wider trunk earns its parameters on segmental metrics (Section 7.2).

The baseline, WindowClassifier in the same file, deliberately shares the stem and inception backbone and differs only in the head: global average pooling over a 128-step window followed by a single linear classifier (40,674 parameters). Sharing the backbone isolates the variable the experiment is about -- the task formulation -- from architecture quality. Its predictions are broadcast back to dense sequences (each window's label repeated 128 times) so the shared scorer can treat it identically to the dense model.

3.3 Data design

The simulator is a measurement instrument, and it is designed like one. Each activity has a profile (fundamental frequency, accelerometer and gyroscope amplitudes, gravity orientation, noise level); each subject has a persistent profile (amplitude scale 0.85-1.18, frequency scale 0.9-1.1, noise scale 0.8-1.3, a constant 6-axis offset, a phase). Sequences of 512 steps contain 2-4 segments with distinct consecutive activities and an 8-step smoothing window at each boundary, so transitions are localised but not infinitely sharp Dirac edges. Subjects are partitioned before any training: 0-11 supply the 300 training sequences (0-9 for tuning fits, 10-11 for tuning validation), 12-15 supply the 100 test sequences. Chapter 5 discusses what this design can and cannot evidence.

3.4 Module map

File	Role	Key contents
data_gen.py	synthetic IMU simulator	PROFILES, subject_params, gen_segment, gen_sequence, generate
model.py	both networks	SEBlock, InceptionBlock, TemporalBlock, DenseHARNet, WindowClassifier, count_params
metrics.py	the shared scorer	macro_f1, change_points, segments, segmental_metrics, evaluate_dense
train.py	protocol driver	boundary_weights, train_dense, train_baseline, stage_tuning, stage_final
demo.py	qualitative check	rle, timeline printout for one unseen test sequence
export_onnx.py	deployment export	torch.onnx.export with dynamic axes, graceful note on failure
MOBILE.md	deployment design	conversion path, streaming inference, on-device personalisation
REAL_DATA_KAGGLE.md	real-benchmark plan	HAPT-first loader plan, splits, expected differences

4. Implementation -- A File-by-File, Function-by-Function Walkthrough

This chapter explains every file and every function a reader needs to reproduce, audit or extend the system. Code excerpts are shortened only by removing boilerplate imports and blank lines; every behaviour described is in the shipped code.

4.1 data_gen.py -- the simulator

PROFILES is a dictionary keyed by activity index 0-5 (walking, walking_upstairs, walking_downstairs, sitting, standing, laying). Each entry fixes a fundamental frequency in Hz (2.0 walking, 1.7 upstairs, 2.3 downstairs, 0.0 for the static classes), accelerometer and gyroscope amplitudes, a gravity direction vector, and a noise standard deviation. Two details carry the realism the experiment needs: walking_downstairs (index 2) has the highest noise (0.07) and, in gen_segment, an impact-spike term; and the three static classes differ almost only in gravity orientation (sitting tilts gravity toward [0, 0.70, 0.71], laying toward [1, 0, 0.10]), which is why Chapter 7 finds them nearly perfectly separable while the dynamic trio confuses.

subject_params(rng) draws one persistent profile per subject: amplitude and frequency scales, noise scale, a 6-dimensional constant offset, and a phase. Persistence is the point: all sequences from a subject share the profile, so subject shift is a real distributional shift the model must cross, not per-sequence noise it can average away.

gen_segment(act, length, rng, subj, t0) synthesises one activity interval. It lays the normalised gravity vector onto the accelerometer channels; for dynamic classes it adds a fundamental sinusoid and its second harmonic across axes with class-specific amplitudes, plus the downstairs impact spikes (a 0.3-amplitude pulse where the fundamental crosses 0.92); for static classes it adds a slow 0.3 Hz sway. Gaussian noise scaled by profile and subject, and the subject offset, are added last.

gen_sequence(rng, subj) assembles a 512-step sequence: it draws 2-4 segments by choosing cut points at least 80 steps from the ends and from each other, assigns activities with no two consecutive equal, generates each segment, concatenates, and then smooths an 8-step window around each cut with a [0.25, 0.5, 0.25] kernel so boundaries are steep ramps rather than single-sample jumps. It returns the float32 signal (512, 6) and the per-timestep label vector.

generate(n, subjects, seed) builds a split: it fixes each subject's profile from a per-subject generator seeded seed + 1000 + subject_id (so profiles are stable and reproducible), then generates n sequences from random subjects in the list. **main()** writes data/train.npz (300 sequences, subjects 0-11, seed 42) and data/test.npz (100 sequences, subjects 12-15, seed 123) in the schema X (N, 512, 6), y (N, 512), subjects (N) -- the exact schema the HAPT loader of REAL_DATA_KAGGLE.md is specified to reproduce.

4.2 model.py -- the networks

SEBlock(ch, r=8) implements Section 2.3: a two-layer perceptron Linear(ch, ch//r), ReLU, Linear(ch//r, ch), Sigmoid applied to the temporal mean of its input, broadcast-multiplied back onto the feature map.

InceptionBlock(in_ch, branch_ch=16, out_ch) builds the four parallel branches (Conv1d with kernels 1, 3, 5, 11, padding k//2, each followed by GroupNorm(4) and ReLU), a projection (Conv1d from 4 x branch_ch to out_ch with kernel 1, GroupNorm(8), ReLU), an SEBlock, and a shortcut (1x1

convolution if channels change, else identity). Its forward concatenates branch outputs, projects, applies SE, adds the shortcut, and applies ReLU. The GroupNorm group counts (4 on 16-channel branches, 8 on the fused width) keep roughly 4-6 channels per group at the widths used.

TemporalBlock(ch, dilation) is one residual dilated layer: Conv1d kernel 3 with padding equal to dilation (so length is preserved), GroupNorm(8), ReLU, added back to its input under a final ReLU.

DenseHARNet(in_ch=6, num_classes=6, width=48, branch_ch=16) wires the stem (Conv1d kernel 5, stride 2, GroupNorm(8), ReLU), two inception blocks, the temporal stack (dilations 1, 2, 4, 8), and the 1x1 head. Its forward runs the encoder at half rate, linearly interpolates the features back to the input length, applies the head, and transposes to (batch, length, classes). The width argument is the only capacity knob the sweep touches, which is what makes the sweep's parameter counts (39,662 at width 32; 68,898 at width 48) a clean function of one variable.

WindowClassifier shares the stem and two inception blocks, then replaces everything downstream with global temporal averaging and a linear head -- the minimal change that converts the dense network into the windowed baseline of Section 3.2.

count_params(model) sums trainable parameter counts; run as a script, model.py prints counts at widths 32/48/64, both output shapes, and the baseline count -- a self-check used during development to catch wiring errors before any training time was spent.

4.3 metrics.py -- the shared scorer

macro_f1(y_true, y_pred, C=6) computes per-class one-versus-rest precision, recall and F1 over flattened timesteps and returns the mean and the per-class list. It is implemented by hand (no scikit-learn anywhere in the project) and unit-checked during development: a perfect prediction yields exactly 1.0.

change_points(a) returns indices where a label sequence changes (np.diff nonzero, shifted by one). **segments(a)** run-length-encodes a sequence into (label, start, end) triples. Both are small; both are load-bearing, because boundary and segmental metrics are defined over exactly these two views of a sequence.

segmental_metrics(Y, P, taus=(10, 25, 50)) implements Section 2.7's segmental F1 and edit score. For each sequence and each threshold it greedily matches predicted segments to unmatched same-label ground-truth segments by IoU, counts matches above the threshold as true positives, and pools counts across sequences before forming F1 (pooling, rather than averaging per-sequence F1, matches the MS-TCN protocol and prevents short sequences from dominating). The edit score uses _lev, a textbook two-row Levenshtein dynamic program over the collapsed segment label lists, normalised per sequence as (1 - distance / max length) x 100 and averaged.

evaluate_dense(predict_fn, X, Y) is the single entry point every model is scored through. It batches prediction, then computes: frame accuracy and macro F1; the transition mask (for each true change point, timesteps from 10 before to 10 after, clipped to the sequence) and accuracy on it, with the masked timestep count; boundary matching at tolerance 10 (each predicted change point matched greedily to the nearest unmatched true one) giving TP/FP/FN, precision, recall and F1; and the segmental metrics. Returning one dictionary with identical keys for every model is what makes results.json a flat, auditable comparison rather than a collage of per-model scripts.

4.4 train.py -- the protocol driver

boundary_weights(y, extra=3.0) builds the weight matrix of Section 2.6: for each sequence it finds the boundary positions, computes every timestep's distance to the nearest boundary by broadcasting, and returns 1 + 3 exp(-distance / 10) as a float tensor.

train_dense(width, use_weights, epochs, Xtr, Ytr, seed, lr=3e-3) trains one dense model: Adam (weight decay 1e-4) with a cosine schedule over the run, batches of 16 sequences (permuted slices of up to 32 indices drawn from the permutation -- the code's slicing quirk is benign because the permutation covers the dataset), per-timestep cross-entropy with reduction none multiplied by the boundary weights (or ones, when use_weights is false), and -- only when use_weights is true -- the smoothing term: the squared difference of adjacent log-probabilities, clamped at 4, averaged, scaled by 0.15, added to the loss. Coupling the two terms behind one flag is what makes configuration B a true ablation of the whole transition-aware package (a limitation discussed in Section 8: the design cannot attribute the gain to one term alone).

train_baseline(epochs, Xtr, Ytr) builds the window dataset by cutting each sequence into four non-overlapping 128-step windows labelled by majority vote, and trains WindowClassifier with plain cross-entropy, Adam at lr 2e-3.

stage_tuning() constructs the subject split that governs all selection decisions: from the training file, subjects below 10 form the sub-train set and subjects 10-11 the validation set. It runs the three configurations of Chapter 6 (skipping any already present in tuning.json, and checkpointing each model to ckpt_NAME.pt as it finishes -- a resumability decision that mattered in practice on a shared CPU machine, where the final stage alone took 1,092.8 seconds).

stage_final() selects the best configuration by validation macro F1, retrains it on all 300 training sequences for 10 epochs with a fresh seed, saves densehar.pt and best_config.json, scores it on the unseen test subjects, then loads the tuning checkpoints for configurations A and B and scores both on the same test set (the test ablation), and finally trains and scores the broadcast window baseline. It assembles results.json in two writes -- once before the baseline (so a baseline failure cannot lose the dense results) and once with the baseline and the windowed-collapse summary block. This two-write pattern, like the tuning checkpoints, is defensive engineering for long CPU runs, and it is documented here because reproducibility includes surviving interruptions.

4.5 demo.py and export_onnx.py

demo.py loads the test file, picks the first sequence with at least three distinct activities, loads DenseHARNet at the width recorded in best_config.json with the densehar.pt weights, and prints the true and predicted timelines in run-length form (the rle helper) plus the sequence's per-timestep accuracy. The example it produces on the shipped checkpoint -- true [0-171] walking_upstairs, [172-319] standing, [320-511] laying; predicted identical except the first boundary lands at 174 instead of 172, for 99.6% per-timestep accuracy on an unseen subject's sequence -- is quoted in Chapter 7 as the qualitative face of the aggregate numbers.

export_onnx.py attempts a torch.onnx.export of the final network with named input imu and output logits and dynamic batch and time axes at opset 14. It catches every exception and, on failure, writes ONNX_EXPORT_NOTE.md stating the error and the fallback routes. In the reference environment the export fails with OnnxExporterError because the onnx package is not installed; the note records the actual error text and the fallbacks (install the package, export a fixed-shape graph, or go through PyTorch Mobile / ExecuTorch). Chapter 8 treats this as a handled, documented gap rather than a hidden one.

5. Dataset

5.1 Why synthetic data, stated plainly

The project uses synthetic data, and this chapter's job is to make that choice legible rather than to apologise for it or to disguise it. Three reasons. First, the research question is about formulation and failure modes -- does dense prediction fix transition collapse, and what new failure does it introduce? -- and a simulator whose ground truth is exact at every timestep, whose transitions are placed by construction, and whose subject variation is a controlled parameter is the right instrument for isolating those questions. Second, real per-timestep transition labels are scarce: the standard HAR benchmarks are window-labelled, and HAPT, the exception, requires the Kaggle path documented in REAL_DATA_KAGGLE.md rather than a local download. Third, everything in the pipeline -- schema, splits, metrics, training stages -- is written against an .npz interface, so the synthetic study is a dress rehearsal whose every number-producing component transfers unchanged to real data.

The cost is equally plain, and it bounds every claim in this report: the simulator's activities are stylised (sinusoidal dynamics, Gaussian noise, idealised gravity), real IMU streams contain placement variation, clothing and device artefacts, transitional actions that are themselves activities, and label noise, none of which are modelled. Accuracy figures here are therefore evidence about the method and the protocol, never about real-world performance, and the report makes no state-of-the-art claim on any real benchmark. What it claims is SOTA methodology: the evaluation protocol the field's segmentation literature would demand.

5.2 Generator statistics and structure

The generator (Section 4.1) produces, per split: 300 training sequences from subjects 0-11 and 100 test sequences from subjects 12-15, each 512 timesteps at a nominal 50 Hz (10.24 seconds), 6 channels, containing 2-4 activity segments with 8-step smoothed boundaries. Class structure is deliberately asymmetric: the three dynamic classes are periodic signals distinguished by frequency (2.0 / 1.7 / 2.3 Hz), amplitude and impact structure, while the three static classes are near-constant signals distinguished by gravity orientation. A reader should therefore expect -- and Chapter 7 confirms -- near-ceiling performance on static classes and errors concentrated in the dynamic trio. Subject variation is multiplicative and additive (amplitude scale 0.85-1.18, frequency scale 0.9-1.1, noise scale 0.8-1.3, constant offsets), persistent within a subject.

5.3 Splits and the leakage rule

Split	Subjects	Sequences	Used for
Tuning sub-train	0-9	(subset of 300)	fitting configurations A, B, C
Validation	10-11	(subset of 300)	all model selection; 1,596 transition timesteps
Final train	0-11	300	retraining the selected configuration
Test	12-15	100	one final evaluation; 3,918 transition timesteps

The rule enforced throughout: no subject contributes to both sides of any decision, and the test set is touched only by stage_final, after the configuration is frozen. This is the LOSO spirit of Section 2.8 with grouped subjects rather than a full leave-one-out rotation -- a deliberate trade of statistical completeness for compute, whose consequence (selection variance across subject groupings) is noted in Chapter 8.

5.4 Limitations of the dataset

Beyond stylisation: transitions are smoothed ramps of a fixed 8-step width, so the generator cannot express the drawn-out, multi-phase transitions of real movement; consecutive segments are constrained only to differ, so some sequences contain transitions (laying directly to walking upstairs) that are biomechanically implausible as adjacency, though each segment individually is plausible; and because static classes are gravity-separated, the dataset cannot measure the sitting-versus-standing confusion that dominates real static-class errors. Each of these is a reason the HAPT run (Chapter 8) is a scientific necessity rather than an optional extra.

6. Experiments

6.1 Protocol

All experiments follow one protocol, implemented by train.py and frozen before the test set was scored:

1. **Tuning stage.** Three configurations are trained for 6 epochs on the sub-train subjects (0-9) and scored on validation subjects (10-11) with the full metric suite. Selection is by validation macro F1, with the other metrics reported for analysis.
2. **Final stage.** The selected configuration is retrained from scratch on all 300 training sequences (subjects 0-11) for 10 epochs with cosine learning-rate decay and a different seed, then scored once on test subjects 12-15.
3. **Test ablation.** The tuning-stage checkpoints of configurations A (transition-aware package on) and B (package off), which share width, epochs and sub-train data, are scored on the same test set, isolating the package's effect at a fixed, smaller training budget.
4. **Baseline.** The window classifier is trained for 4 epochs on majority-labelled 128-step windows from the same training sequences, broadcast to dense predictions, and scored by the same function on the same test set.

Optimisation throughout: Adam, initial learning rate 3e-3 (dense) or 2e-3 (baseline), weight decay 1e-4, cosine schedule, batch size 16 sequences (dense) / 64 windows (baseline), PyTorch default CPU threading, fixed seeds (0 for tuning, 1 for the final). The final stage's recorded wall time is 1,092.8 seconds on the shared CPU host.

6.2 The sweep: configurations A, B, C

Config	Width	Params	Boundary weighting	Smoothing loss	Epochs	What it tests
A	48	68,898	yes (1 + 3 exp(-d/10))	yes, lambda 0.15	6	the full transition-aware package
B	48	68,898	no	no	6	ablation: plain per-timestep cross-entropy
C	32	39,662	yes	yes	6	capacity: does a 42% smaller model suffice?

The sweep is small by design and honest about it: with CPU-only training, three configurations chosen to answer three specific questions (package on/off; width up/down) are worth more than a wide grid whose winner would be selected on noise. Validation subjects are disjoint from both the sub-train and the test subjects, so selection cannot leak in either direction.

6.3 Baseline construction

The baseline is the strongest fair representative of the incumbent formulation, not a straw man: it shares DenseHAR's stem and inception backbone (Section 3.2), trains on the same sequences, and its window length (128) and majority labelling follow standard windowed-HAR practice, matching the window scale at which InceptionKAN-class models are usually evaluated. Broadcasting its window labels to dense sequences is precisely what deploying a windowed model on a stream does, so scoring the broadcast with dense metrics is a measurement of deployment reality, not a rhetorical device.

7. Results and Analysis

Every number in this chapter is copied from results.json or tuning.json (Appendix A reproduces results.json in full). Percentages are rounded to two decimals; the underlying values are exact.

7.1 The headline: transition collapse, quantified

Metric (test, unseen subjects 12-15)	DenseHAR (final)	Window-128 baseline (broadcast)
Per-timestep accuracy	95.35%	85.71%
Macro F1	0.9545	0.8573
Transition-window accuracy (plus/minus 10)	90.63%	48.03%
Boundary F1 (tolerance 10)	0.525	0.174
Boundary precision / recall	0.360 / 0.974	0.196 / 0.156
Segmental F1@10 / @25 / @50	0.621 / 0.607 / 0.572	0.899 / 0.892 / 0.855
Edit score	74.50	85.92
Parameters	68,898	40,674

Read the table in two halves. On frame-level and boundary-level measures, the dense model wins decisively, and the margin is largest exactly where the theory of Section 1.1 says it should be: inside transition windows the baseline scores 48.03% -- statistically indistinguishable from guessing between the two activities a transition window contains -- while DenseHAR holds 90.63%. The drops from each model's own overall accuracy are the cleanest statement of the result: the baseline falls 37.68 points at transitions (85.71 to 48.03); DenseHAR falls 4.71 points (95.35 to 90.63). The windowed formulation does not merely underperform at transitions; it collapses there, and the collapse is a property of the formulation, since the baseline shares the dense model's backbone and training data.

The second half of the table is the counter-result, given its own full section (7.5) rather than a footnote: on segmental F1 at all three thresholds and on edit score, the baseline wins by wide margins. Sections 7.2 to 7.4 first establish how the model was selected and what the components contribute; Section 7.6 analyses the normalisation failure; Section 7.7 collects the qualitative evidence.

7.2 Tuning: frame accuracy saturates, segmental metrics select

Config	Val accuracy	Val macro F1	Val transition acc	Val boundary F1	Val seg F1@25	Val edit score
A (width 48, package on)	99.32%	0.9928	91.60%	0.849	0.882	85.16
B (width 48, plain CE)	99.13%	0.9907	89.72%	0.814	0.857	81.64
C (width 32, package on)	99.16%	0.9910	89.85%	0.782	0.840	77.74

The spread in validation frame accuracy across the three configurations is 0.19 points; in macro F1, 0.21 points. Had selection been run on accuracy -- the default habit imported from windowed HAR -- the three models would have been effectively tied, and the choice among them arbitrary. The segmental and boundary metrics separate them cleanly and consistently: configuration A leads on every single measure, B is second on most, and the narrower C trails on edit score by 7.4 points behind A despite being within 0.16 accuracy points of it. The mechanism is Section 2.7's: frame

metrics average over the 96% of timesteps that are mid-segment and easy, while segmental metrics integrate over precisely the boundary behaviour in which the configurations differ. Configuration A was selected (by the pre-registered rule, validation macro F1, on which it also leads), and the final test result -- A-derived, retrained on more data -- confirms the selection was not a validation artefact: the retrained model improves on every tuning-stage number when moved to the harder unseen-subject test.

Two further observations belong to a careful reading. All three configurations reach boundary recall 1.000 on validation with precision between 0.642 and 0.738 -- the asymmetry that dominates the test results (Section 7.5) is already visible in embryo on validation subjects, a point Chapter 8 returns to. And the validation-to-test generalisation gap for the final model (99.32% validation accuracy for its configuration against 95.35% test accuracy) is the expected signature of subject shift: validation subjects are held out from fitting but the configuration was selected on them, while test subjects influenced nothing. Reporting both numbers, rather than only the flattering one, is the protocol working as intended.

7.3 Ablation: the transition-aware package earns its place, modestly

The test ablation compares the tuning-stage A and B checkpoints -- same width, same 6-epoch budget, same sub-train subjects -- on the unseen test set:

Metric (test, 6-epoch sub-train models)	A: boundary weighting + smoothing	B: plain cross-entropy	Difference
Accuracy	94.21%	93.99%	+0.22
Macro F1	0.9435	0.9415	+0.0020
Transition-window accuracy	86.52%	84.48%	+2.04
Boundary F1	0.456	0.452	+0.004
Segmental F1@25	0.533	0.529	+0.004
Edit score	65.71	64.61	+1.09

The package helps everywhere and dramatically nowhere, and the pattern of its help is the informative part: the gain is an order of magnitude larger on transition-window accuracy (+2.04 points) than on overall accuracy (+0.22), which is the signature expected if the mechanism works as designed -- the boundary weighting spends model capacity exactly in the transition band. The final model (same package, full training data, 10 epochs) then improves every ablation number substantially (95.35% accuracy, 90.63% transition accuracy, edit score 74.50), showing that training-set size and schedule length contributed more than the loss package did. An honest hierarchy of effects for this system, from these numbers: formulation (dense versus windowed) first, data and training length second, the transition-aware loss third. Because the package's two terms are coupled behind one flag (Section 4.4), the experiment cannot separate boundary weighting from smoothing; that separation is a listed future experiment, not a claim.

7.4 Per-class analysis: the dynamic trio carries the errors

Class	DenseHAR test F1	Baseline test F1	Validation F1 (config A)
walking	0.883	0.799	0.978
walking_upstairs	0.910	0.790	0.994
walking_downstairs	0.941	0.892	0.990
sitting	0.999	0.895	0.998
standing	0.996	0.870	0.997
laying	0.999	0.897	0.999

The static classes are essentially solved by DenseHAR on unseen subjects (F1 at or above 0.996): gravity orientation is a subject-stable cue, and the dense model reads it at every timestep. The baseline's static-class F1 (0.87-0.90) is notably worse -- not because it cannot recognise statics, but because its broadcast quantises static intervals to window boundaries, corrupting the edges of every static bout. The dynamic trio is where DenseHAR's remaining error lives (0.883-0.941), and the validation column shows why: on validation subjects the same classes scored 0.978-0.994, so the test drop is subject shift in gait dynamics -- amplitude and frequency profiles of unseen walkers -- not a capacity ceiling. This is the same phenomenon InceptionKAN reported in its LOSO evaluation, reproduced here in dense form, and it motivates the on-device personalisation direction of Chapter 8: the cue that varies by subject is exactly the cue a few minutes of wearer-labelled data could recalibrate.

7.5 The honest counter-result: over-segmentation hands the segmental metrics to the baseline

This section reports, in full, the result that cuts against the project's headline, because understanding it is worth more than the headline.

The numbers. On the unseen test set the baseline beats DenseHAR on segmental F1@10 (0.899 against 0.621), F1@25 (0.892 against 0.607), F1@50 (0.855 against 0.572) and edit score (85.92 against 74.50). These are not marginal gaps. By the segmentation literature's own headline metrics, the windowed baseline is the better segmenter on unseen subjects, while being far worse per-timestep and at boundaries. Both statements are true simultaneously; the rest of this section explains how, and what follows.

The mechanism. Boundary counts expose it. The test set contains 192 true change points. DenseHAR predicts 520 change points: 187 true positives, 333 false positives, 5 false negatives -- precision 0.360, recall 0.974. It finds essentially every real transition (missing only five) and announces 333 transitions that do not exist: short spurious islands and doubled boundaries inside what should be single segments. Each spurious island does triple damage in segmental accounting: it splits a true segment so the pieces' IoU with the whole falls, it adds an unmatched predicted segment (a false positive), and it inserts an extra label into the collapsed sequence, raising the Levenshtein distance. The edit-score arithmetic shows it directly: DenseHAR's mean edit distance is 3.34 operations per sequence, the baseline's is 0.49.

The baseline's opposing profile is parsimony by construction. Broadcast over 128-step windows, it can emit at most 4 segments per 512-step sequence, and its predicted change points number only 153 (30 true positives, 123 false positives, 162 false negatives; precision 0.196, recall 0.156). It misses most real boundaries -- recall 0.156 is a model that barely sees transitions at all -- but the segment

sequences it does emit are short and usually in the right order, which is what F1-with-IoU and edit score reward.

Why DenseHAR over-segments specifically on unseen subjects. On validation subjects the same configuration's boundary precision was 0.738 with zero false negatives; on test subjects precision falls to 0.360 while recall stays at 0.974. The model did not become less sensitive on new subjects; it became less specific. The interpretation consistent with all the numbers is that mid-segment evidence on unseen walkers (whose amplitude and noise scales sit outside the tuning distribution) occasionally crosses the decision boundary for a few timesteps, and a per-timestep classifier with no mechanism to price segment duration converts each such excursion into two boundary announcements. The smoothing loss suppresses exactly this, and the ablation shows it helps, but a fixed lambda tuned on subjects 10-11 is evidently too weak for subjects 12-15. Nothing in the frame-level metrics warns about any of this: accuracy drops only 4 points from validation to test while boundary precision halves. A project reporting only accuracy would have shipped the failure invisibly.

What follows. The segmentation literature's standard remedy is architectural, not a bigger hammer on the same loss: MS-TCN's multi-stage design feeds the first stage's per-timestep probability sequence into refinement stages trained to remove spurious segments while preserving true boundaries, and MS-TCN++ and boundary-aware refinement work refine the same idea. That is the specified next step (Chapters 8 and 9), with two cheaper interim mitigations also listed: validation-tuned median filtering of predictions (a duration prior applied post-hoc) and per-subject head-only personalisation. The counter-result therefore does positive work in the research programme: it converts "dense is better" from a slogan into a precise claim -- dense prediction fixes transition collapse, and its remaining defect is a measured over-segmentation with a known remedy and a metric suite that will verify whether the remedy worked.

7.6 Error analysis: the BatchNorm prototype at about 53%

During development, an early prototype of the encoder -- which used BatchNorm in place of GroupNorm and had a narrower temporal receptive field -- reached only about 53% validation accuracy on held-out subjects, against 99.32% for the final design on the same validation split. The figure is reported in the project README, and this report repeats it with two qualifications stated openly: the 53% comes from an early prototype run, not from a controlled configuration in the final sweep (it does not appear in tuning.json, and no number in this chapter other than this one is outside results.json/tuning.json); and two changes were made together -- normalisation and receptive field -- so the prototype comparison cannot attribute the recovery to either alone. What can be done, and what this section does, is to lay out the mechanism (Section 2.5) that makes BatchNorm the prime suspect and to state the controlled experiment that would settle it.

The mechanism, briefly restated: with 16-sequence batches drawn from few subjects, BatchNorm's per-batch statistics are subject statistics; running averages accumulated over training subjects mis-normalise unseen subjects at every layer simultaneously, and small placement or offset differences compound multiplicatively through depth. GroupNorm's per-sample statistics have no cross-subject pathway at all. The receptive-field change, by contrast, predicts a different failure signature -- confusion between rhythm-similar classes, spread across segment interiors -- whereas a normalisation failure predicts broadly distorted features and depressed accuracy across all classes at once. The observed 53% (far above the 16.7% six-class chance level, far below any usable model) fits the second signature better: a model whose features are systematically distorted by mis-normalisation, rather than one that merely lacks context. The decisive experiment costs little:

retrain configuration A with GroupNorm swapped for BatchNorm, all else identical, on the same split. It is listed first among the extension exercises in Chapter 10 precisely because this project leaves it open.

The generalisable lesson is the one a viva examiner is most likely to probe, so it is stated as a principle: in wearable ML, every design choice that pools statistics across people is a subject-shift liability until proven otherwise -- normalisation layers, global feature standardisation fitted on training subjects, and batch-dependent operations of any kind. DenseHAR's data pipeline standardises nothing globally for the same reason.

7.7 Qualitative evidence

Aggregate metrics are corroborated by the demo's run-length timelines. On the demo sequence from unseen subject 13, the true segmentation [0-171] walking_upstairs, [172-319] standing, [320-511] laying is reproduced with the first boundary displaced by two timesteps (predicted 174) and the second exact -- 99.6% per-timestep accuracy, with boundary behaviour of the kind the boundary metrics describe. The failure mode of Section 7.5 is equally visible when it occurs: sequences where the prediction inserts short islands (typically of a dynamic class inside a static segment, or of walking inside a walking_upstairs segment) are exactly the sequences whose edit distances dominate the mean of 3.34. The qualitative and quantitative pictures agree, which is itself a small validity check on the metric implementation.

7.8 Summary of findings

1. Dense formulation removes the transition collapse: minus 4.71 points at transitions against the baseline's minus 37.68, at comparable model size (68,898 against 40,674 parameters).
2. Frame accuracy saturates in tuning (99.13-99.32% across configurations) and cannot select models; segmental F1 and edit score separate them and selected the configuration that test confirmed.
3. The transition-aware loss package gives a small, consistent, transition-concentrated gain (+2.04 transition accuracy points in the controlled ablation), smaller than the gain from more training data and a longer schedule.
4. Errors concentrate in the dynamic trio and are attributable to subject shift in gait profiles, mirroring InceptionKAN's LOSO behaviour.
5. Over-segmentation on unseen subjects (boundary precision 0.360 at recall 0.974) hands segmental F1 and edit score to the baseline -- the project's most important negative result and the definition of its next stage.
6. Batch statistics are a subject-shift hazard in HAR: the BatchNorm prototype's collapse to about 53% is analysed mechanistically, with the isolating experiment specified but not yet run.

8. Discussion, Limitations and the Scale-Up Path

8.1 What the results do and do not establish

The results establish three things with reasonable firmness, within the synthetic regime: the windowed formulation has a structural transition defect that dense prediction removes; the segmentation metric suite is necessary to see either that defect or dense prediction's own defect; and subject shift, not capacity, is the binding constraint on generalisation for this model class. They do not establish anything about real-world accuracy, about the ranking of dense versus windowed methods on real data (where transitions are noisier and the windowed baseline's segmental advantage may grow or shrink), or about the isolated contributions of boundary weighting versus smoothing, or of GroupNorm versus receptive field. Chapter 7 has flagged each open attribution at the point where the number appears; this chapter collects the threats systematically.

8.2 Threats to validity

Synthetic data (external validity). The dominant threat, discussed in Section 5.4. Sinusoidal activity models cannot express real transition morphology, device artefacts or label noise. Mitigation: none within this study; the HAPT path of Section 8.4 exists precisely to retire this threat.

Selection on two validation subjects. Configurations were selected on subjects 10-11 only; with two subjects, selection could be lucky. The test result (a different, larger subject group, where the selected configuration's retrained version performed as the tuning order predicted) reduces but does not eliminate the concern. A full LOSO rotation would.

Coupled ablation. The transition-aware package ablates two losses together; BatchNorm versus GroupNorm is confounded with receptive field. Both are stated openly and both have specified isolating experiments.

Single seed per configuration. Tuning runs use seed 0 and the final seed 1. Seed variance at this accuracy level is unlikely to reorder the configurations on segmental metrics (gaps of 2.5-7.4 edit points), but it has not been measured, and a multi-seed rerun is a cheap strengthening.

Greedy segment matching. Segmental F1 here uses greedy best-IoU matching, as does the reference MS-TCN evaluation code; optimal bipartite matching could differ marginally. Because every model is scored by the same code, comparisons are internally valid even if absolute values shift slightly under a different matcher.

8.3 Mobile deployment

MOBILE.md specifies the deployment design, summarised here. The model is small (68,898 parameters), purely convolutional over a 6-channel stream, and needs no GPU: a single forward pass on a 512-step buffer is a few milliseconds on a modern phone CPU, and INT8 dynamic quantisation would put the weights well under 500 KB. Streaming uses a ring buffer of 512 samples at 50 Hz with a hop of 25-50 samples; only the newest hop's labels are emitted, bounding latency to the hop rather than the buffer. Display smoothing (majority vote over recent labels) is specified for the user interface only, never for stored transition timestamps, because smoothing would erase exactly the boundaries the system exists to localise. The export path runs through export_onnx.py to ONNX, then ONNX Runtime Mobile or onnx2tf to TFLite; in the reference environment the export currently stops at the missing onnx package and writes ONNX_EXPORT_NOTE.md with the error and fallbacks (install the package; fixed-shape export; ExecuTorch). The honest status is therefore: deployment designed and costed, export scripted with a documented blocker, on-device latency not yet measured -- a measurement the HAPT phase should produce on real hardware rather than estimate.

One deployment idea follows directly from Section 7.4's analysis: because the residual errors are subject-profile effects, a head-only on-device personalisation step -- fine-tuning only the final 1x1 classifier on a few minutes of the wearer's labelled data, encoder frozen -- is the cheapest plausible recovery of the validation-to-test gap, and it keeps raw data on the device.

8.4 The real-data path: HAPT on Kaggle

REAL_DATA_KAGGLE.md packages the conversion of this study into a real-benchmark study, and its key decisions are worth restating because they were made with this project's earlier Kaggle experience in mind. The primary dataset is HAPT (UCI id 341): 30 subjects, phone accelerometer and gyroscope at 50 Hz, the same six basic activities plus six postural transitions, with per-timestep labels in RawData/labels.txt -- no label reconstruction needed, which removes the largest error source in dense relabelling. UCI-HAR (id 240) is the secondary source, its dense labels reconstructed approximately from window order. The notebook attaches the Kaggle mirror of the dataset and, if the mirror is not wired into the working directory, symlinks it in before the loader runs, because direct UCI downloads are unreliable on the Kaggle network path -- a lesson carried over from the author's earlier WISDM probing work. Sequences are sliced per subject and experiment into 512-step windows containing at least one transition; the six postural-transition classes are reported both as their own labels (a 12-class problem) and mapped to the following activity (the 6-class problem comparable to this report). Splits are by subject (train 21, validation 4, test 5), model.py, metrics.py and the training stages are reused unchanged behind the same .npz schema, and the full sweep plus final plus baseline is budgeted at under 30 minutes on a T4. Only numbers from that run may be quoted as real-benchmark results; the expected differences, stated in advance so they cannot be retrofitted, are messier transitions, higher upstairs/downstairs confusion, and worse over-segmentation before the smoothing weight is re-tuned on the validation subjects.

8.5 The refinement stage: attacking over-segmentation

Section 7.5 defines the next model change precisely. A refinement stage in the MS-TCN sense takes the first stage's per-class probability sequence (512 x 6) as input -- not the raw sensors -- and applies a small dilated temporal network trained with the same classification plus smoothing losses to output a cleaned sequence. Refinement stages work because the errors they must fix are visible in the probability sequence itself: a two-step island has a characteristic signature (a brief, low-confidence excursion flanked by a confident majority class) that is far easier to detect in probability space than in raw signal space. The success criterion is pre-registered by this report's own tables: raise boundary precision from 0.360 toward the validation level without surrendering recall below roughly 0.95, and close at least half the segmental F1@25 gap to the baseline (0.607 toward 0.892) while holding transition-window accuracy at or above 90%. Two cheaper baselines must be beaten first and are therefore part of the same experiment: median filtering of predicted labels with a validation-tuned width, and raising the smoothing weight lambda. If a median filter matches the refinement stage, the honest conclusion is that the duration prior was the whole story, and the report of that experiment should say so.

9. Conclusion and Future Work

DenseHAR set out to test a formulation, and the formulation test succeeded in both directions. Replacing one-label-per-window with one-label-per-timestep removed the structural transition collapse of windowed HAR -- a 37.7-point fall reduced to 4.7 points -- at a model size (68,898 parameters) compatible with on-phone inference, using an encoder inherited from the author's InceptionKAN line so the comparison indicts the formulation rather than the features. The same study, by adopting the segmentation literature's full metric suite, caught the dense model's own structural weakness with equal clarity: on unseen subjects it over-segments, and a parsimonious windowed baseline legitimately wins segmental F1 and edit score. A report that contained only the first finding would be advertising; containing both, it is a research result with a next step attached.

Future work, in the order this report's evidence ranks it:

1. **The isolating normalisation experiment** (BatchNorm versus GroupNorm, all else fixed) to convert Section 7.6's mechanism into a measurement.
2. **The refinement stage** of Section 8.5, against the median-filter and lambda-raising controls, scored on this report's pre-registered criteria.
3. **The HAPT real-benchmark run** of Section 8.4, unchanged protocol, both the 6-class and 12-class label mappings.
4. **Loss decoupling**: boundary weighting and smoothing ablated separately, and lambda re-tuned per split to test the subject-dependence hypothesis of Section 7.5.
5. **Full LOSO rotation and multi-seed tuning** to retire the selection-variance threat of Section 8.2.
6. **More sensors and personalisation**: magnetometer and barometer channels (the generator's profile structure extends naturally), and the head-only on-device personalisation experiment of Section 8.3, measured on real hardware with the ONNX path unblocked.

---

10. Learning Guide

10.1 Concept checklist

A reader who can explain each item below, with the formula or the mechanism, owns the material of this report:

- Windowed versus dense task formulation, and the structural error cap inside a transition window
- Change points, segments, and run-length encoding of label sequences
- Multi-scale (inception) temporal convolution and why kernel sizes 1/3/5/11 span step and rhythm scales
- Squeeze-and-excitation channel gating, and the role of the residual shortcut
- Dilated convolution and receptive-field arithmetic for dilations 1, 2, 4, 8
- BatchNorm versus GroupNorm under subject shift (batch statistics are subject statistics)
- Boundary-weighted cross-entropy: w_t = 1 + 3 exp(-d_t / 10), and why the scale matches the evaluation tolerance
- MS-TCN truncated smoothing loss and why truncation protects true boundaries
- Frame accuracy, macro F1, and what their averaging hides
- Transition-window accuracy and the transition mask
- Boundary precision/recall/F1 at a matching tolerance, and the over-segmentation fingerprint (low precision, high recall)
- Segmental F1 at IoU thresholds 10/25/50, and why parsimony wins it
- Edit score via Levenshtein distance over collapsed label sequences
- LOSO-style subject-disjoint evaluation and leakage through subject identity
- Half-rate encoding as an efficiency/resolution trade priced against the evaluation tolerance
- Over-segmentation: mechanism, metric signature, and the MS-TCN refinement remedy

10.2 Viva questions with model answers

Q1. Your windowed baseline scores 85.71% overall. Why do you call it collapsed? Because the same model, scored with the same code on the same test sequences, scores 48.03% within plus or minus 10 timesteps of a true transition -- a 37.68-point fall. Inside a transition window the single-label constraint caps dense accuracy (a window split evenly between two activities caps at roughly 50%), so the baseline's transition score sits at that structural cap. Overall accuracy conceals this because transition timesteps are only 3,918 of 51,200 test timesteps.

Q2. Is the comparison fair? The baseline was not designed to output dense labels. It is fair in the only sense that matters for deployment: broadcasting window labels is what using a windowed model on a stream means. The baseline also shares DenseHAR's stem and inception backbone and trains on the same sequences, so the comparison isolates the formulation. And the report does not hide the direction in which the comparison is unfair to DenseHAR: segmental F1 and edit score reward the baseline's structural parsimony, and it wins both (Section 7.5).

Q3. Why does frame accuracy fail to separate your tuning configurations? Because about 96% of timesteps are mid-segment, where all reasonable models agree. The configurations differ at

boundaries and in segment integrity, which frame accuracy dilutes by a factor of roughly 25. The numbers: accuracy spread 0.19 points across A/B/C, while edit score spread is 7.4 points and boundary F1 spread is 6.7 points. Model selection must use a metric whose averaging matches the phenomenon of interest.

Q4. Define segmental F1@25 and explain how a model with worse frame accuracy wins it. Collapse predictions and ground truth to segments. A predicted segment is a true positive if an unmatched ground-truth segment with the same label overlaps it with IoU above 0.25. Over-segmentation manufactures false positives (extra pieces that cannot match) and degrades IoU (pieces of a whole match the whole poorly), so a model that emits few, correctly ordered segments -- the broadcast baseline, at most 4 per sequence -- can beat a per-timestep-better model that emits 520 boundaries where 192 exist. DenseHAR's segmental F1@25 is 0.607 against the baseline's 0.892, despite a 9.6-point frame-accuracy advantage.

Q5. Read DenseHAR's boundary precision and recall like a diagnostician. Recall 0.974 with precision 0.360 means the model misses almost no real transition (5 false negatives of 192) but two of every three boundaries it announces are spurious. That asymmetry is over-segmentation, localised to unseen subjects: on validation subjects precision was 0.738 at recall 1.000. The defect is specificity under subject shift, not sensitivity, which is why the proposed remedies are duration-aware (refinement stage, median filter, stronger smoothing) rather than capacity increases.

Q6. Why GroupNorm? What exactly goes wrong with BatchNorm in HAR? A HAR batch contains few subjects, and a subject's sequences are highly correlated, so batch means and variances are effectively subject statistics. The network adapts to batch-dependent normalisation that vanishes at test time, where frozen running averages describe the average training subject and mis-normalise every layer for a new wearer. GroupNorm normalises within each sample over channel groups, removing the cross-subject pathway. The project's early BatchNorm prototype reached about 53% validation accuracy against 99.32% for the GroupNorm design; the report is careful to note the receptive field changed in the same step, and names the isolating experiment as future work rather than claiming the attribution is proven.

Q7. Derive the boundary weighting and defend its constants. $w_t = 1 + 3 \exp(-d_t / 10)$, d_t the distance to the nearest true boundary. A boundary timestep counts 4x a mid-segment one, because boundary timesteps are rare (under 8% here), hardest, and the point of the study. The decay scale 10 matches the evaluation tolerance of plus or minus 10 steps: emphasis is concentrated exactly where the boundary metrics will judge the model. The ablation shows the package (weighting plus smoothing) buys +2.04 transition-window accuracy points against +0.22 overall -- evidence the emphasis lands where intended.

Q8. Why truncate the smoothing loss? Untruncated smoothing penalises the squared log-probability change between adjacent timesteps without limit, so it taxes genuine transitions -- where adjacent distributions should differ sharply -- hardest of all. Truncating at 4 caps the per-step penalty: flicker, which is many small penalties, still accumulates a cost across channels and steps, while one decisive true transition pays the cap once. It is the difference between discouraging jitter and discouraging change.

Q9. Why encode at half rate? What does it cost? The stride-2 stem halves the sequence length every downstream layer processes, roughly halving compute and doubling effective receptive field per layer. The cost is boundary quantisation: the head's features are computed on a 2-step grid and upsampled, so a boundary can be misplaced by at most about 2 input steps. Priced against the

plus-or-minus-10 evaluation tolerance, that is a 5x safety margin -- resolution was spent only where the metric cannot detect it. If the application demanded sample-exact boundaries (some clinical gait measures do), the trade would need revisiting.

Q10. Your test accuracy (95.35%) is below validation (99.32%). Is the model overfit? It is subject shift, which is related but distinct. The configuration was selected on validation subjects 10-11, so validation is mildly optimistic by selection; test subjects 12-15 influenced nothing. Per-class F1 shows the drop concentrates in the dynamic trio (walking 0.978 validation to 0.883 test) while static classes hold at 0.996-0.999 -- gait-profile variation across people, the same effect InceptionKAN's LOSO evaluation showed. The proposed mitigations are subject-directed (personalisation, refinement), not the generic anti-overfitting toolkit.

Q11. Why not just median-filter the predictions and skip the refinement stage? That is the right challenge, and the project treats the median filter as a control the refinement stage must beat, not as a dismissed alternative. A median filter imposes a minimum segment duration; it will remove islands shorter than half its width, including genuine short activities, and it cannot use confidence information. The refinement stage operates on probabilities and can in principle distinguish a confident short segment from an unconfident flicker. If the experiment shows the filter matching the learned stage, the honest conclusion is that duration was the whole story -- Chapter 8 pre-commits to reporting that outcome.

Q12. What is the edit score, and why report it alongside F1? The edit score collapses each sequence to its ordered segment labels, computes the Levenshtein distance to the true label sequence, and normalises: (1 - distance / max length) x 100. It measures whether the activity story is right -- correct activities in correct order -- ignoring timing entirely. Segmental F1 prices timing overlap; edit score prices narrative; frame accuracy prices instants. DenseHAR's edit score (74.50, mean distance 3.34) against the baseline's (85.92, 0.49) is the purest statement of the over-segmentation cost, stripped of timing arguments.

Q13. Defend the synthetic dataset to a sceptical examiner. On three grounds: the ground truth is exact per timestep, which real window-labelled benchmarks do not provide; subject variation is a controlled, persistent parameter, so subject shift can be induced and measured rather than hoped for; and every number-producing component consumes an .npz schema that the HAPT loader reproduces, so the study is a protocol rehearsal, not a dead end. The concession is equally explicit: no real-benchmark claim is made, stylised transitions and gravity-separated static classes bound the external validity, and the HAPT run is specified as the required next step, with its expected difficulties stated in advance.

Q14. Why is HAPT the right real dataset for this project specifically? Because it is the standard HAR dataset whose annotation is natively dense and transition-aware: 30 subjects, accelerometer and gyroscope at 50 Hz matching this project's input specification in channels and rate, the same six basic activities, and six labelled postural transitions with per-timestep spans in labels.txt. Most HAR benchmarks would require reconstructing dense labels from window labels -- introducing exactly the boundary uncertainty the project exists to measure. HAPT lets the protocol of Chapter 6 run unchanged.

Q15. If you had one more week of compute, what single experiment most strengthens this thesis? The HAPT run, because it retires the largest threat to validity (synthetic data) rather than polishing internal numbers. Within the synthetic regime, the BatchNorm isolation experiment is the highest-value small run, since Section 7.6's mechanism is currently argued, not measured. Both answers follow from ranking experiments by the size of the uncertainty they remove, not by the size

of the number they might produce.

10.3 Extension exercises

1. **Isolate the normalisation effect.** Retrain configuration A with GroupNorm replaced by BatchNorm, everything else fixed. Report the full metric suite on validation and test, and test Section 7.6's prediction that the damage concentrates under subject shift rather than uniformly.
2. **Decouple the losses.** Run the 2x2 ablation (boundary weighting on/off x smoothing on/off) at the tuning budget, and check whether the transition-window gains add, overlap, or interact.
3. **Median-filter control.** Grid-search a post-hoc median filter width on validation subjects; report test segmental F1, edit score and boundary precision. This is the control the refinement stage must beat.
4. **Build the refinement stage.** Implement a two-stage model (probability-sequence input, dilated stack, same losses) and score it against the pre-registered criteria of Section 8.5.
5. **Multi-seed tuning.** Rerun configurations A/B/C with three seeds and report means and spreads for every metric; check whether the A-over-B segmental ordering is seed-stable.
6. **The HAPT run.** Execute REAL_DATA_KAGGLE.md end to end and place the resulting table beside Table 7.1, with a paragraph on every place reality disagreed with the synthetic study's expectations.

---

11. How to Run

All commands run from the densehar project directory. Dependencies are PyTorch and NumPy only (see the project's requirements.txt); no scikit-learn is used anywhere.

```
# 1. Generate the synthetic dataset (data/train.npz, data/test.npz)
python data_gen.py

# 2. Full protocol: tuning sweep, final retrain, test ablation, baseline
python train.py            # or stage by stage:
python train.py tuning     # configurations A/B/C -> tuning.json, ckpt_*.pt
python train.py final      # -> densehar.pt, best_config.json, results.json

# 3. Qualitative check: true vs predicted timeline on one unseen sequence
python demo.py

# 4. Mobile export attempt (writes densehar.onnx, or ONNX_EXPORT_NOTE.md
#    explaining the blocker, if the onnx package is unavailable)
python export_onnx.py
```

Sanity checks a new user should expect: model.py run as a script prints parameter counts (39,662 / 68,898 / and the width-64 count) and both output shapes; a perfect prediction through metrics.evaluate_dense returns accuracy 1.0, segmental F1 1.0 at all thresholds and edit score 100. Training is CPU-feasible but not instant: the final stage recorded 1,092.8 seconds on the project's shared host, and the tuning stage is checkpointed per configuration, so an interrupted sweep resumes where it stopped.

12. References

1. Sahoo, A. and Ramanujam, E. InceptionKAN: Multi-Scale Temporal Kolmogorov-Arnold Networks for Parameter-Efficient Human Activity Recognition. Preprint submitted to Pervasive and Mobile Computing, 2026. (Author's prior work; windowed results quoted as context: 97.25% UCI-HAR, 98.45% PAMAP2, 95.54% WISDM v2.)

2. Farha, Y. A. and Gall, J. MS-TCN: Multi-Stage Temporal Convolutional Network for Action Segmentation. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019. (Architecture pattern, smoothing loss, and the segmental F1 / edit evaluation protocol adopted here.)

3. Li, S. J., AbuFarha, Y. and Gall, J. MS-TCN++: Multi-Stage Temporal Convolutional Network for Action Segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022. (Refinement-stage direction proposed in Chapter 8.)

4. Wu, Y. and He, K. Group Normalization. European Conference on Computer Vision (ECCV), 2018.

5. Hu, J., Shen, L. and Sun, G. Squeeze-and-Excitation Networks. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

6. Szegedy, C. et al. Going Deeper with Convolutions. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015. (Inception multi-scale branching.)

7. Bai, S., Kolchinsky, J. Z. and Koltun, V. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271, 2018. (Dilated temporal convolutions.)

8. Anguita, D. et al. A Public Domain Dataset for Human Activity Recognition Using Smartphones. European Symposium on Artificial Neural Networks (ESANN), 2013. (UCI-HAR.)

9. Reyes-Ortiz, J.-L. et al. Transition-Aware Human Activity Recognition Using Smartphones. Neurocomputing, 2016. (The HAPT dataset and its per-timestep transition labels.)

10. Ordonez, F. J. and Roggen, D. Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition. Sensors, 2016. (DeepConvLSTM; subject-shift behaviour in HAR.)

11. Bulling, A., Blanke, U. and Schiele, B. A Tutorial on Human Activity Recognition Using Body-Worn Inertial Sensors. ACM Computing Surveys, 2014.

12. Ioffe, S. and Szegedy, C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. International Conference on Machine Learning (ICML), 2015. (The normalisation analysed critically in Sections 2.5 and 7.6.)

Appendix A -- Full results.json

The complete contents of the project's results.json at the time of writing. tuning.json contains the same three tuning_runs objects; best_config.json contains {"width": 48, "name": "A_width48_boundary"}.


```
{
  "protocol": "SOTA methodology on SYNTHETIC data only: tuning on held-out subjects 10-11, final
 test on unseen subjects 12-15. No real-benchmark SOTA claim.",
  "params_densehar_best": 68898,
  "params_baseline_window": 40674,
  "best_config": {"name": "A_width48_boundary", "width": 48, "boundary_weighted": true,
"epochs": 6},
  "final_epochs_full_train": 10,
  "train_seqs": 300,
  "test_seqs": 100,
  "training_seconds_final_stage": 1092.8,
  "densehar_final_test": {
    "accuracy": 0.9534765625, "macro_f1": 0.9545498118833632,
    "per_class_f1": [0.8827, 0.9097, 0.9406, 0.9989, 0.9960, 0.9994],
    "transition_window_accuracy": 0.9063297600816743, "transition_timesteps": 3918,
    "boundary_f1": 0.5252808988764045, "boundary_precision": 0.3596153846153846,
    "boundary_recall": 0.9739583333333334,
    "boundary_tp": 187, "boundary_fp": 333, "boundary_fn": 5,
    "segmental_f1@10": 0.6206140350877193, "segmental_f1@25": 0.6074561403508771,
    "segmental_f1@50": 0.5723684210526315,
    "edit_score": 74.4974238520594, "edit_distance_mean": 3.34
  },
  "baseline_window128_broadcast_test": {
    "accuracy": 0.857109375, "macro_f1": 0.8573101352120563,
    "per_class_f1": [0.7992, 0.7902, 0.8917, 0.8952, 0.8704, 0.8971],
    "transition_window_accuracy": 0.48034711587544665, "transition_timesteps": 3918,
    "boundary_f1": 0.17391304347826086, "boundary_precision": 0.19607843137254902,
    "boundary_recall": 0.15625,
    "boundary_tp": 30, "boundary_fp": 123, "boundary_fn": 162,
    "segmental_f1@10": 0.8990825688073395, "segmental_f1@25": 0.891743119266055,
    "segmental_f1@50": 0.855045871559633,
    "edit_score": 85.91666666666666, "edit_distance_mean": 0.49
  },
  "windowed_collapse_at_transitions": {
    "baseline_overall_accuracy": 0.857109375,
    "baseline_transition_window_accuracy": 0.48034711587544665,
    "densehar_transition_window_accuracy": 0.9063297600816743
  }
}
```


Per-class F1 arrays are in the activity order of data_gen.ACTIVITIES: walking, walking_upstairs, walking_downstairs, sitting, standing, laying. The two test ablation objects (ablation_test_subtrain6ep_width48_boundary and ablation_test_subtrain6ep_width48_plain) and the three tuning_runs objects are reproduced in the tables of Sections 7.2 and 7.3; their full precision values are in results.json and tuning.json in the project directory.

Appendix B -- File Listing

File	Description
data_gen.py	Synthetic IMU generator: activity and subject profiles, 512-step multi-segment sequences, subject-disjoint splits
model.py	DenseHARNet (inception + SE + dilated stack + dense head) and WindowClassifier baseline; parameter counting
metrics.py	Shared scorer: macro F1, boundary P/R/F1, MS-TCN segmental F1@10/25/50, edit score
train.py	Protocol driver: boundary weights, tuning sweep (stages), final retrain, test ablation, baseline
demo.py	Run-length true-versus-predicted timeline for one unseen test sequence
export_onnx.py	ONNX export with dynamic axes; graceful ONNX_EXPORT_NOTE.md fallback
MOBILE.md	Mobile deployment design: conversion path, streaming inference, quantisation, personalisation
REAL_DATA_KAGGLE.md	HAPT-first real-benchmark plan for Kaggle, with splits and expected differences
ONNX_EXPORT_NOTE.md	Recorded export status in the reference environment (onnx package missing) and fallbacks
README.md	Project summary with the headline and counter-result tables
data/train.npz	300 sequences, subjects 0-11 (generated by data_gen.py)
data/test.npz	100 sequences, subjects 12-15 (generated)
tuning.json	The three tuning configurations with full validation metric objects
best_config.json	{"width": 48, "name": "A_width48_boundary"}
results.json	All reported numbers (see Appendix A)
densehar.pt	Final model weights (width 48, 10 epochs, full training subjects)
ckpt_A_width48_boundary.pt / ckpt_B_width48_plain.pt / ckpt_C_width32_boundary.pt	Tuning-stage checkpoints used for the test ablation
baseline_window.pt	Window baseline weights
requirements.txt	PyTorch + NumPy

Appendix C -- Real-Data Addendum (HAPT)

Added after the main report was written. The main chapters above evaluate DenseHAR on the synthetic simulator of Chapter 5, and they say so throughout. This addendum reports the same model, the same metric suite and the same subject-disjoint discipline on real data -- the HAPT dataset -- and it supersedes the synthetic results as the project's headline. Every number below is copied from `results_real.json` *in the project directory; the full narrative version is* `REAL_DATA_RESULTS.md` *alongside it.*

C.1 Why this addendum exists

The main report's headline -- 95.35% per-timestep accuracy, 90.63% inside transition windows -- was measured on data produced by the author's own generator. Chapter 5.4 already states the limitation plainly: a simulator's subjects differ from one another only in the parameters the author chose to vary, so a model can score highly by learning the generator rather than the phenomenon. The synthetic run proved the pipeline, the protocol and the metric suite. It could not, by construction, evidence real-world accuracy. The experiment that could -- sketched as future work in Section 8.4 -- has now been run, and this addendum is its record, including the part where the original design did not beat its baseline.

C.2 Dataset and protocol

Dataset. HAPT (Reyes-Ortiz et al., Reference 9; UCI Machine Learning Repository id 341): 30 subjects, waist-mounted smartphone, accelerometer and gyroscope at 50 Hz, with per-timestep labels in `RawData/labels.txt`. Twelve classes: the six basic activities of Chapter 5 (walking, walking upstairs, walking downstairs, sitting, standing, laying) plus six postural transitions (stand-to-sit, sit-to-stand, sit-to-lie, lie-to-sit, stand-to-lie, lie-to-stand). The source archive's integrity was verified before use.

Loader. `load_hapt.py` builds, per (experiment, user) stream, 512-step windows at stride 256, matching the sequence length of the synthetic pipeline so that `model.py` and `metrics.py` are reused unchanged. Windows touching an unlabelled sample are dropped (27.36% of the 1,122,772 raw samples are unlabelled in HAPT). Windows containing an activity boundary or any transition-class timestep are always kept; pure single-activity windows are capped at at most 30% of each split. Two label versions are stored: `y6`, the six-class labelling with each transition class mapped to its *target* activity (stand-to-sit to sitting, sit-to-stand to standing, sit-to-lie to laying, lie-to-sit to sitting, stand-to-lie to laying, lie-to-stand to standing), and `y_trans`, a boolean mask of the timesteps whose original label was a transition class, which enables the transition-class timestep accuracy reported below -- a metric with no windowed analogue.

Split and selection discipline. The split is fixed and subject-disjoint, with no overlap (asserted in code): train 21 subjects [1, 3, 5, 6, 7, 8, 11, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26], validation 4 subjects [27, 28, 29, 30], test 5 subjects [2, 4, 9, 10, 12]. Test subjects are drawn from the official HAPT test-subject list; validation subjects are carved from the official train list. Normalisation uses per-channel mean and standard deviation computed from the train subjects' streams only. All model, hyperparameter and post-processing selection used the validation split (macro F1 primary). The test split was evaluated exactly six times in the entire programme -- once per final candidate, always after selection -- and all 22 logged experiments, including failures, are preserved in `results_real.json`.

Split	Subjects	Sequences	Transition timesteps	Sequences with at least one boundary
Train	21	862	18.6%	445
Validation	4	145	18.7%	78
Test	5	177	18.9%	97

Transition-class timesteps are 8.24% of all labelled raw HAPT samples; the boundary-enriched sequencing raises their share to about 19% of evaluated timesteps, so this evaluation is deliberately transition-heavy -- the region of the stream this research line exists to measure. One structural property matters for reading the results: no test window contains more than two basic activities, because HAPT experiments are single-bout recordings; 124 of the 177 test windows contain transition-class timesteps.

C.3 Phase 1: the original configuration on real data -- a tie

Phase 1 ran the report's selected configuration unchanged (width 48, boundary-weighted cross-entropy plus truncated smoothing loss at lambda 0.15, 14 epochs, validation-selected checkpoint; 68,898 parameters) against the window-128 broadcast baseline of Chapter 6 (10 epochs, validation-selected; 40,674 parameters), both trained on the 862 training sequences.

Test metric	DenseHAR (Phase 1)	Window-128 baseline
Per-timestep accuracy	0.7886	0.7891
Macro F1	0.7253	0.7258
Transition-window accuracy (plus or minus 10 steps)	0.5955	0.4787
Accuracy on transition-class timesteps	0.8571	0.7730
Accuracy on non-transition timesteps	0.7726	0.7929
Boundary F1 (precision / recall)	0.1353 (0.078 / 0.516)	0.0874 (0.093 / 0.083)
Segmental F1@10 / @25 / @50	0.410 / 0.388 / 0.355	0.775 / 0.749 / 0.693
Edit score	63.42	75.52

The diagnosis, in the model's own numbers, has four parts. First, **the headline metrics tie**: 0.7886 against 0.7891 accuracy, 0.7253 against 0.7258 macro F1. Whatever the synthetic chapters demonstrate, the unmodified design does not beat a windowed baseline on real data. Second, **the transition advantage is real but narrow**: plus 11.7 points at transition windows and plus 8.4 points on transition-class timesteps, while ordinary timesteps are *worse* than the baseline (0.7726 against 0.7929). The reformulation of Section 1.2 buys exactly what it promises and, in this configuration, nothing else. Third, **over-segmentation, a nuisance on synthetic data, dominates on real data**: 592 false boundaries against 50 true positives (precision 0.078), which is why segmental F1@25 collapses to 0.388 against the baseline's 0.749 and the edit score to 63.42 against 75.52. The failure mode analysed in Section 7.5 is the same one, amplified by real signal variability. Fourth, **the dominant problem is cross-subject generalisation**: both models score validation macro F1 of about 0.92 and test macro F1 of about 0.73, a gap of roughly 19 points. The four validation subjects are much easier than the five test subjects, and upstairs collapses on unseen subjects (F1 0.386, against 1.000 on validation) while standing is weak (0.616) -- the static-pair problem of Section 7.4 in a harsher form. Capacity is not the binding constraint; subject shift is.

C.4 Phase 2: improvement levers, screened on validation only

Phase 2 was guided by a literature brief (kept in the project as `IMPROVEMENT_RESEARCH.md`: MS-TCN/ASRF refinement and post-processing, physics-respecting augmentation, class-weighting evidence, and negative evidence on focal loss and boundary-weighted cross-entropy). Each candidate was screened on validation macro F1; only validation winners were promoted to a single test evaluation. Total training compute for all 22 logged experiments was about 22 minutes on four CPU threads.

Experiment	Lever(s)	Val accuracy	Val macro F1	Seg F1@25	Edit
p1_dense (Phase 1)	configuration A, unchanged	0.8940	0.9189	0.641	75.2
p2_plain	plain cross-entropy (no boundary weights, no smoothing)	0.9004	0.9225	0.628	76.0
p2_long	configuration A, 30 epochs	0.9088	0.9287	0.625	75.6
p2_cw	median-frequency class weights	0.8871	0.8966	0.561	67.5
p2_refine	MS-TCN refinement stage	0.8984	0.9396	0.768	83.6
p2_f11	gravity/magnitude feature channels (11 ch)	0.8966	0.9418	0.716	79.5
p2_c12	12-class supervision	0.9122	0.9517	0.684	81.8
p2_aug	physics augmentation	0.9265	0.9584	0.705	79.4
p2_aug_c12	augmentation + 12-class supervision	0.9372	0.9648	0.719	80.4
p2_aug_c12_refine	augmentation + 12-class + refinement	0.9340	0.9625	0.824	85.6

Four findings organise the screening. **Physics augmentation is the single biggest lever** (validation F1 0.9189 to 0.9584): accelerometer and gyroscope are rotated jointly by one random rotation (plus or minus 40 degrees, applied in raw units so the physics stays consistent), with smooth magnitude warping, scaling, small time-warps and jitter, on the training split only. It attacks the validation-to-test subject gap of Section C.3 directly, exactly as the literature predicted. **Twelve-class supervision is the second lever** (0.9517 alone, 0.9648 combined with augmentation): giving the transition classes their own labels teaches the model what a transition *is*; at scoring time, probabilities are mapped back to six classes by summing over each target activity's preimage. **Post-processing is the segmental fix**: applied to the frozen Phase-1 model, a validation-tuned pack (Gaussian smoothing sigma 8, minimum segment duration theta 25) moved validation segmental F1@25 from 0.641 to 0.866 and boundary F1 from 0.277 to 0.443 at unchanged architecture -- a pure decoding gain with zero retraining, and the direct remedy for the over-segmentation quantified in Section C.3. **The refinement stage buys structure more than frames**: the best raw-model segmental F1@25 (0.768 alone, 0.824 combined) at a smaller frame-accuracy gain, again as the MS-TCN literature suggests. Finally, **seed variance is large** -- validation F1 from 0.932 to 0.965 across three seeds of the winning configuration -- which is the honest reason the final system ensembles three seeds rather than reporting the luckiest one.

The final system was selected as the candidate with the highest validation macro F1 (0.9675): physics augmentation plus 12-class supervision, a three-seed probability ensemble, and validation-tuned post-processing (sigma 8 Gaussian, theta 15 minimum duration; theta of at most 15 samples is 0.3 s, safely below the roughly 1 s duration of real HAPT transitions). A co-leading candidate -- the same recipe with an MS-TCN refinement stage, single seed -- ranked fourth on validation F1 (0.9634) and is

therefore reported as co-leading rather than headline, even though its test scores are the best of the programme on several metrics; the headline was chosen on validation, before test was touched, and the discipline is worth more than the difference.

C.5 Final test results

Test metric	Final system (headline)	With refinement stage (co-leading)	Window-128 baseline
Per-timestep accuracy	0.9038	0.8990	0.7891
Macro F1	0.9398	0.9415	0.7258
Transition-window accuracy	0.6093	0.6246	0.4787
Accuracy on transition-class timesteps	0.8744	0.8946	0.7730
Accuracy on non-transition timesteps	0.9106	0.9000	0.7929
Boundary F1	0.4087	0.5045	0.0874
Segmental F1@10 / @25 / @50	0.836 / 0.822 / 0.788	0.858 / 0.840 / 0.819	0.775 / 0.749 / 0.693
Edit score	84.88	86.23	75.52

Read against Section C.3, the arc is the result. The final system beats the identically-evaluated baseline on **every** headline metric, by 10 to 22 points: accuracy plus 11.5, macro F1 plus 21.4, transition-window accuracy plus 13.1, transition-class timestep accuracy plus 10.1, segmental F1@25 plus 7.3, edit score plus 9.4. The two deficits that defined Phase 1 -- the segmental collapse and the non-transition deficit -- are both closed (non-transition accuracy 0.9106 against the baseline's 0.7929; segmental F1@25 0.822 against 0.749). Transition windows remain intrinsically hard for every model: the baseline falls 31.0 points from its overall accuracy inside them, the final system 29.5 points, but the dense formulation preserves a 13-to-15-point absolute advantage exactly there, which is the claim Chapter 1 set out to test.

Two worked examples from the final ensemble on test subject 4 make the behaviour concrete. In sequence 1 (truth: laying for steps 0-221, then sitting to 511), the prediction switches at step 244, scoring 95.7% -- inside the annotated transition span (steps 222-431), which is physically sensible and a reminder that the `y6` convention places the boundary at the transition's first timestep. In sequence 0, a transition truncated at the window edge (truth: laying 0-477, sitting 478-511) is missed entirely, yet the cost is bounded at 6.6% of the window; edge-truncated transitions are a known cost of fixed-length sequencing, disclosed rather than hidden.

C.6 Per-class analysis

Class	Baseline	Phase 1	Final system	With refinement
walking	0.889	0.893	1.000	1.000
walking_upstairs	0.444	0.386	1.000	1.000
walking_downstairs	0.732	0.777	0.994	1.000
sitting	0.721	0.739	0.853	0.849
standing	0.623	0.616	0.815	0.853
laying	0.945	0.941	0.976	0.946

Augmentation essentially **solved the dynamic classes** on unseen subjects -- upstairs F1 from 0.386 to 1.000 -- because rotating and warping the training signals multiplies the effective variety of gaits

far more than it multiplies postures. What remains is the known hard core of waist-worn HAR: **sitting versus standing** (F1 0.82-0.85), separable mainly by gravity direction and micro-sway, and the pair every HAPT system struggles with. Gravity/magnitude channels alone helped validation (0.9418) but did not beat augmentation, and their combination with augmentation (validation F1 0.9563) did not beat augmentation plus 12-class supervision either -- feature engineering lost to data augmentation and supervision design on this dataset, which is itself a finding worth recording.

C.7 Negative results, kept

Three negatives from this programme deserve the same prominence as the wins, because each contradicts a plausible prior -- two of them contradict this report's own main chapters. **Median-frequency class weights hurt**: validation F1 0.8966, the worst of all screening runs; re-weighting toward the rare dynamic classes damaged the dominant statics that fill most timesteps. **Boundary-weighted cross-entropy gave nothing on real data**: the full configuration-A loss scored validation F1 0.9189 against plain cross-entropy's 0.9225, and the smoothing loss alone matched the combination -- the mechanism that Sections 2.6 and 7.3 credit on synthetic data does not transfer, consistent with the negative evidence in the Phase-2 literature brief and now confirmed on HAPT. **Augmentation alone did not raise transition-class timestep accuracy** (0.8506 raw, against Phase 1's 0.8571); that metric was lifted by post-processing (0.8595) and above all by the refinement stage (0.8946). One engineering failure is also logged for completeness: the first augmentation-plus-features run crashed because the augmentation assumed six channels; it was fixed by augmenting the physical channels and recomputing derived features from the augmented signal.

C.8 Comparisons, read with their protocols

Against the synthetic results of the main chapters (DenseHAR 95.35%, baseline 85.71%): the synthetic task was easier in every way that matters -- balanced classes, clean per-subject profiles, transitions drawn from the same generator as training. Real HAPT Phase 1 landed at 78.9% for *both* models. The synthetic chapters stand as a pipeline and protocol proof; their numbers are superseded by this addendum and should not be quoted as accuracy results.

Against the windowed HAPT reference (TinyHAR, from the author's earlier work in the same research line: 90.45% window accuracy, 0.794 macro F1, 12-class): the final system's per-timestep accuracy (0.9038) sits in the same band, but the protocols differ in ways that make direct ranking meaningless. The scoring units differ (one majority decision per 128-sample window, with transition timesteps absorbed into the majority, against *every* timestep, in an evaluation where about 19% of timesteps sit at or inside transitions); the label sets differ (12-class window labels against 6-class dense labels with transitions mapped to their target, plus transition-timestep scoring that has no windowed analogue); and the subject splits, though both subject-disjoint, are not the same lists. The dense macro F1 of 0.94 exceeding the windowed 0.794 is likewise a protocol artefact (6 against 12 classes), not a claim of superiority. The honest comparison is the one inside this addendum: the same window-128 architecture, retrained and broadcast under *this* protocol, scores 0.7891 / 0.7258.

C.9 Limitations

Four limitations bound these results. **Small subject counts**: four validation and five test subjects, with seed variance alone at plus or minus 3 validation F1 points; the numbers would move under a different fixed split and are reported for *this* split, which is fixed and documented. **The validation subjects are easier than the test subjects** (both models: validation F1 about 0.92 against test about 0.73 in Phase 1); selection on an easier validation split is a real methodological weakness of this run -- the augmentation gains were large enough to survive it, but finer distinctions, such as refinement against ensemble, should be read as provisional. **Windows contain at most two basic activities**, a property of HAPT's single-bout recordings rather than of the model; long multi-activity streams, the actual deployment setting, remain untested on real data. **The `y6` boundary convention** (a transition counts as its target activity from its first timestep) makes boundary-timing metrics pessimistic by up to a transition length; segmental F1 and edit score are the fairer structural measures here. Sitting versus standing, at F1 of about 0.82-0.85, remains the open problem.

C.10 Reproduce

```
python3 load_hapt.py            # hapt.zip -> data_hapt.npz (+ stats json)
python3 train_real.py phase1    # configuration A + baseline -> results_real.json
python3 train_real.py exp --name p2_aug_c12 --epochs 20 --augment --classes 12 --no-test
python3 train_real.py postproc --model model_real_p2_aug_c12.pt --name final_augc12_postproc
python3 train_real.py ensemble --models
model_real_p2_aug_c12.pt,model_real_p2_aug_c12_s1.pt,model_real_p2_aug_c12_s2.pt --name
final_ens_augc12 --postproc-from final_augc12_postproc
```

Files: `load_hapt.py`, `train_real.py`, `data_hapt.npz`, `data_hapt_stats.json`, `results_real.json` (all 22 experiments with full metric suites), checkpoints `model_real_*.pt`, and the run logs `train_real_p1.log` and `train_real_p2*.log`, all in the project directory.

End of report.